My 2D c++ engine rundown:

Hi, here is a quick rundown on the capabilities and how to use my engine:

This engine was built on the SFML library.

1: General structure:
This engine is designed to be similar to unity. You can create game objects, which you can then add components to using something like this:

```
Gameobject* ParalaxHandler = new Gameobject();
ParalaxScrollingHandler* psh = ParalaxHandler->AddComponent<ParalaxScrollingHandler>();
```

Game objects can have many components but only one of the same type per game object, and they are automatically managed by engine when they are no longer in use.

Game objects can also have other child game objects, child objects inherit the position of their parent object, they can also be used to add additional colliders to objects that may need a more complex collider setup.

```
Scene:
(GameObject) Empty
    |--- (GameObject) Empty      (Component) [class ParalaxScrollingHandler]
              |--- (GameObject) Spacebg      (Component) [class SpriteRendererComponent]  (Component) [class AnimatorComponent]
              |--- (GameObject) Spacebg      (Component) [class SpriteRendererComponent]  (Component) [class AnimatorComponent]
```

You can easily create your own components by creating a new c++ file and header, including component, then making a class that inherits the component class. However, when creating your own components, you must include a cloning function, or else your component will use the base cloning function and may lose functionality or crash the project.

2: Physics objects and collisions:
Everything physics will be located in the Collider, GameEssentials and Rigidbody files, this engine uses bounding boxes to calculate collisions, it also uses sweep and prune to cut down on the number of collision calculations per frame.

Rigidbodies can have their velocity set and the game object they are attached to will move accordingly; they do not have a way to control acceleration yet.

You can easily create trigger colliders instead of the regular ones, which generate collision events when overlapped by an object that is not excluded from the trigger or vice versa.

Example of collider + rigidbody setup:

```
Collider* bpcollider = BulletPea->AddComponent<Collider>();
bpcollider->Layer = "Player";
bpcollider->ExcludedLayers = { "Player", "LevelBounds" };
BulletPea->AddComponent<Rigidbody>();
bpcollider->SetupCollider();
bpcollider->IsTrigger = true;
```

3: Rendering and sprites:

This engine also handles all the rendering work, it is simple to create and use sprites and text. Sprites can be given a layer and sprites that are on a lower layer will be rendered behind sprites on a higher layer. All renderable components inherit from the Renderable class, by default, this engine has a few renderable objects. These are:
Sprites
Text

Example sprite setup:

```
sf::Texture* texture = SceneRh->InitTexture("Spaceship", "Assets/Spaceship.png");
sf::IntRect rect({ 0,0 }, { 23,11 });
SpriteRendererComponent* src = Test->AddComponent<SpriteRendererComponent>(texture, rect);
AnimatorComponent* amc = Test->AddComponent<AnimatorComponent>();
```

On top of having sprites, this engine also supports text rendering for UIs

```
TextRenderer* testtext = BeamText->AddComponent<TextRenderer>(MainFont, "BEAM", 20);

testtext->text.setStyle(sf::Text::Bold);

testtext->text.setFillColor({ 148, 132, 240 });
testtext->text.setCharacterSize(12);


Gameobject* UiCanvas = new Gameobject();

UiCanvasComponent* UiCanvasComp = UiCanvas->AddComponent<UiCanvasComponent>();

BeamText->SetParent(UiCanvas);
```

To use text, you must define your font and have it in your current resource handler:

```
ResourceHandler* SceneRh = &GameEssentialsGlobals::Rh;

sf::View view({ 0,0 }, { 384, 256 });


sf::Font* MainFont = SceneRh->GetFont("Assets/r-type.ttf");
MainFont->setSmooth(false);
```

You must also remember to run the canvas setup function whenever you finish making changes to the canvas:

```
UiCanvasComp->SetupCanvas();
```

4: Animations:

This engine also supports basic sprite animations:

```
SpriteRendererComponent* SpaceSprite = SpaceBg->AddComponent<SpriteRendererComponent>(SpaceTex, SpaceRect, 0);
AnimatorComponent* SpaceAnimator = SpaceBg->AddComponent<AnimatorComponent>();
sf::IntRect RectF1S({ 0,0 }, { 1152,256 });
sf::IntRect RectF2S({ 0,256 }, { 1152,256 });
sf::IntRect RectF3S({ 0,256 * 2 }, { 1152,256 });
sf::IntRect RectF4S({ 0,256 * 3 }, { 1152,256 });
AnimationFrame amSpace0 = { RectF1S, 30 };
AnimationFrame amSpace1 = { RectF2S, 30 };
AnimationFrame amSpace2 = { RectF3S, 30 };
AnimationFrame amSpace3 = { RectF4S, 30 };
Animationstrip amsSpace = { {amSpace0,amSpace1,amSpace2,amSpace3}, true };
//-----------------------------------------------------------

//Adds the anim and plays
SpaceAnimator->AddAnimation(amsSpace, "idle");
SpaceAnimator->PlayAnim("idle");
```

Animations work by moving the rendered rectangle of your spritesheet, you have to declare the rects manually, after that you can make your animation frames, which consist of the rect and the time it takes to transition to the next frame, after that you can make the animation strip, which consists of a list of frames and a bool to determine if it's a looping animation.

5: Inputs and events:

I have also created a custom input and event system which allows you to easily and cleanly set up inputs and events for your game.

You can do this by using the component->bindevent function

```
std::function<void(InputArgs IA)> OnDPressedfn = [this](InputArgs IA)
    {
        this->OnDPressed(IA);
    };

std::function<void(InputArgs IA)> OnSpacePressedfn = [this](InputArgs IA)
    {
        this->OnSpacePressed(IA);
    };

this->bindEvent(sf::Keyboard::Key::W, OnWPressedfn);
this->bindEvent(sf::Keyboard::Key::A, OnAPressedfn);
this->bindEvent(sf::Keyboard::Key::D, OnDPressedfn);
this->bindEvent(sf::Keyboard::Key::S, OnSPressedfn);
this->bindEvent(sf::Keyboard::Key::Space, OnSpacePressedfn);
```

The bind event function allows for both inputs and regular events, these are also handled properly when the component that created these events is deleted.

6: Scene switching:

The main gameloop is controlled in main, however scenes are managed elsewhere, use the scenes files for some quick examples. You can create a new scene class, copying the template scene and adding your own game objects, switching scenes is quite simple, all of the hard stuff is done by the engine behind the scenes, all you need to do is:

```
std::function<void(InputArgs IA)> OnWPressedfn = [this](InputArgs IA)
    {
        //GameEssentialsGlobals::Renderwindow->close();
        NextScene = new SceneTest();
        GameEssentialsGlobals::LoadScene(NextScene);
    };
```

Using the GameEssentialsGlobals, you can load scenes easily, loading a scene will automatically switch you to the new scene.

7: Resources:

This engine also uses a resource handler to reduce on pointless memory usage, holding sound buffers, textures, prefabs and fonts, this is easy to use and quite simple:

```
sf::Texture* SpaceTex = SceneRh->InitTexture("Spacebg", "Assets/SpaceBackground1Animated.png");
```

if this texture already exists in the resource handler, then it will grab a reference to the texture and return it instead of making a new one in memory.

8: Sound:

Sound is quite easy to utilise in my engine, you can create a sound object and play it within a component, or you can attach a sound component to an object and play the sound through that. SoundComponents can hold many sounds and the sounds can be played through the name of the sound.

```
std::unique_ptr<Sound> ExSound = std::make_unique<Sound>("Assets/explosionSound.wav");
ExplosionTest->AddComponent<SoundComponent>()->AddSound(std::move(ExSound), "default");
```

```
Explosion->GetComponent<SoundComponent>()->PlaySound("default");
```

9: Events:

Events are also a core part of this engine, they allow components to communicate with each other easily, one event could trigger many components, and many components can trigger an event. Events are cleaned up after the component is destroyed, meaning that the programmer does not have to worry about manually disconnecting events when destroying game objects.

```
Setup1 = true;
std::function<void(EventArgs)> OnBeamChargeStart = [this](EventArgs)
    {
        this->Held = true;
        //std::cout << "HELD!!!";
    };
std::function<void(EventArgs)> OnBeamChargeEnd = [this](EventArgs)
    {
        this->Held = false;
        FullLastFrame = false;
        GameObject->GetComponent<AnimatorComponent>()->PlayAnim("idle");
        CurrentProgress = 0;
    };
this->bindEvent("ShootStart", OnBeamChargeStart);
this->bindEvent("ShootEnd", OnBeamChargeEnd);
```

10: Timers:

Timers allow for delayed & sequential events to occur, Timers can be programmed in components, timers do not have to be deleted manually by the developer

```
std::function<void()> A = [this](void)
    {
        std::cout << "TimerStuff!!";
        this->GetGameObject()->Destroy();
    };
AddTimer(A, 3, false, false);
```

11: Buttons:

Buttons allow for simple player mouse and click input, you can override the base button class to make your own specialized buttons. There are different bindable events related

to buttons:

```
void BindClickEvent(std::string Handle, std::function<void(MouseEventFireStruct)> fn) { ... }
void UnbindClickEvent(std::string Handle) { ... }
void BindUnClickEvent(std::string Handle, std::function<void(MouseEventStopStruct)> fn) { ... }
void UnbindUnClickEvent(std::string Handle) { ... }
void BindHoverEvent(std::string Handle, std::function<void()> fn) { ... }
void UnbindHoverEvent(std::string Handle) { ... }
void BindUnHoverEvent(std::string Handle, std::function<void()> fn) { ... }
void UnbindUnHoverEvent(std::string Handle) { ... }
void BindRightClickEvent(std::string Handle, std::function<void(MouseEventFireStruct)> fn) { ... }
void UnbindRightClickEvent(std::string Handle) { ... }
void BindRightUnClickEvent(std::string Handle, std::function<void(MouseEventStopStruct)> fn) { ... }
void UnbindRightUnClickEvent(std::string Handle) { ... }
```

You can bind the events anywhere as the bind functions are publicly accessible.

The example below is bound inside of the scene:

```
Button->BindClickEvent("Click1",[Button](UiButtonComponent::MouseEventFireStruct MouseEvent) {

    //Do function!


});
```